

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

**Reinforcement Learning with Verifiable Rewards for Chart Question
Answering**

Sumaiya Ahmed

210041223

Nafisa Haque

210041231

Jannatul Fardus Rakhi

210041237

Department of Computer Science and Engineering

Islamic University of Technology

April, 2026

**Reinforcement Learning with Verifiable Rewards for Chart Question
Answering**

Sumaiya Ahmed

210041223

Nafisa Haque

210041231

Jannatul Fardus Rakhi

210041237

Department of Computer Science and Engineering

Islamic University of Technology

April, 2026

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by **Sumaiya Ahmed**, **Nafisa Haque**, and **Jannatul Fardus Rakhi** under the supervision of **Syed Rifat Raiyan**, Lecturer, Department of Computer Science and Engineering and co-supervision of **Dr. Hasan Mahmud**, Professor, Department of Computer Science and Engineering and **Dr. Md. Kamrul Hasan**, Professor, Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Syed Rifat Raiyan

Lecturer

Department of Computer Science and Engineering

Islamic University of Technology (IUT)

Date: April 22, 2026

Sumaiya Ahmed

Student ID: 210041223

Date: April 22, 2026

Dr. Hasan Mahmud

Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT)

Date: April 22, 2026

Nafisa Haque

Student ID: 210041231

Date: April 22, 2026

Dr. Md. Kamrul Hasan

Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT)

Date: April 22, 2026

Jannatul Fardus Rakhi

Student ID: 210041237

Date: April 22, 2026

*Dedicated to our supervisors, for their guidance at every
stage of this work*

Contents

1	Introduction	1
1.1	Task Definition	2
1.2	Background	2
1.2.1	Reinforcement Learning and Rollouts	2
1.2.2	Policy Update Strategies	3
1.3	Research Objectives	3
1.4	Research Challenges	4
1.5	Research Questions	5
1.6	Contributions	5
1.7	Organization	6
1.8	Why Charts Are a Distinct Visual Reasoning Problem	6
1.9	Motivation for Verifiable Rewards	7
1.10	Scope of the Present Work	7
2	Related Works	9
2.1	Benchmarks and Evaluation	9
2.2	Instruction Tuning and Supervised Fine-Tuning	10
2.3	Agentic and Tool-Use Approaches	10
2.4	Reinforcement Learning for Reasoning	11
2.4.1	Training-Time RLVR	11
2.4.2	Negative Sample Reward and RLVR-Decomposed	12
2.4.3	Correctness of Reasoning, Not Just Answers	12
2.4.4	Test-Time Reinforcement Learning	12
2.4.5	Structured Reasoning and Domain Transfer	13
2.5	Summary	13
2.6	Positioning of This Thesis	13
2.7	Comparison of Method Families	14
2.8	Open Problems Identified from Prior Work	15

3	Proposed Methodology	17
3.1	Overview	17
3.2	Base Model and Parameter-Efficient Fine-Tuning	18
3.3	Structured Output Format and Rollout Generation	19
3.4	Rollout Count: Intended Design and Current Constraint	20
3.5	Parsing and Filtering	21
3.5.1	Answer Normalization	21
3.5.2	Table Representation	21
3.5.3	Reasoning Step Extraction	22
3.6	Reward Components	22
3.6.1	Chart-RVR Base Rewards	22
3.6.2	Hierarchical Correct-Path Consistency (HCPC)	23
3.6.3	Why HCPC Is Hierarchical	24
3.6.4	Correct-Path Filtering	24
3.6.5	Coherence as an Evaluation Signal	24
3.7	Policy Update Strategies	25
3.7.1	GRPO	25
3.7.2	NSR	25
3.8	Training Configuration	26
4	Results and Discussion	27
4.1	Experimental Setup	27
4.1.1	Hardware and Software Environment	27
4.1.2	Training Dataset	27
4.1.3	Evaluation Benchmark and Protocol	28
4.2	Main Results	30
4.3	Analysis	30
4.3.1	RLVR Provides Consistent Improvement Over the Baseline	30
4.3.2	HCPC Improves Rollout-Level Reliability	31
4.3.3	NSR Transfers Successfully to Vision-Language Models	31
4.3.4	HCPC vs. NSR: Explicit vs. Implicit Diversity Promotion	31
4.3.5	Per-Chart-Type Analysis	32
4.3.6	Reasoning Diversity	32
4.3.7	All-Four-Rollouts-Correct Rate	32
4.3.8	Format Compliance	33
4.3.9	Accuracy Distribution	33
4.4	Out-of-Distribution Evaluation on ChartFC	34
4.5	Sample Difficulty Analysis	35

4.6	Hard Problem Types	35
4.7	Why Diversity Did Not Improve Strongly	37
4.8	Limitations	37
4.9	Threats to Validity	38
4.9.1	Internal Validity	38
4.9.2	External Validity	38
4.9.3	Construct Validity	38
4.9.4	Statistical Conclusion Validity	39
5	Conclusion	40
5.1	Summary of What We Did	40
5.2	Summary of Results	41
5.3	Limitations	41
5.4	Future Work	41
5.5	Concluding Remarks	42

List of Figures

3.1	Overview of the HCPC-RLVR training pipeline. A chart image and question enter the system; the proposed model generates $K = 8$ rollouts; each rollout is parsed into chart type, table, reasoning, and answer; the fully-correct subset feeds HCPC; and the aggregated reward drives the policy update. Current experiments use $K = 4$ rollouts because of compute constraints.	18
3.2	Comparison of full-parameter and LoRA fine-tuning. In LoRA, the adapter matrices are much smaller than the frozen base weights, which keeps the effective parameter count low.	19

List of Tables

2.1	Positioning of HCPC-RLVR relative to major method families in chart reasoning and RLVR	15
3.1	Training configuration shared across all HCPC-RLVR variants	26
4.1	Chart type distribution in the 1,000-sample training subset	28
4.2	Comparison between the full Chart-RVR training set and the first 1,000-sample subset used in this thesis	28
4.3	Chart type distribution in the 500-sample ChartQA evaluation set . . .	29
4.4	Evaluation metrics and their definitions	29
4.5	Performance evaluation on ChartQA (in-distribution, 500 samples). Bold indicates the best value per column among all methods.	30
4.6	Out-of-distribution evaluation on ChartFC (500 samples). The comparison is diagnostic because OOD evaluation was completed for GRPO and GRPO+HCPC only.	34
4.7	ChartQA sample difficulty categories from the predefense analysis . .	35
4.8	Common hard problem types observed in low-scoring samples	36
5.1	Summary of key findings	41

List of Abbreviations

CoT	Chain-of-Thought
CQA	Chart Question Answering
GRPO	Group Relative Policy Optimization
HCPC	Hierarchical Correct-Path Consistency
KL	Kullback-Leibler
LoRA	Low-Rank Adaptation
NSR	Negative Sample Reward
OOD	Out-of-Distribution
QA	Question Answering
RL	Reinforcement Learning
RLVR	Reinforcement Learning with Verifiable Rewards
SFT	Supervised Fine-Tuning
VLM	Vision-Language Model
VQA	Visual Question Answering

Acknowledgement

We are grateful to our supervisor, Syed Rifat Raiyan, whose direction kept this project grounded from its earliest conceptual stages through the final evaluation runs. His willingness to engage with the technical details of reward design and his patience when training results raised more questions than they answered made a genuine difference to the quality of this work.

We thank Dr. Hasan Mahmud and Dr. Md. Kamrul Hasan for their guidance on evaluation methodology and for consistently pushing us to think carefully about what the numbers actually mean, not just whether they were large.

We are also grateful to the Systems and Software Lab for providing the computational resources and collaborative environment that made this research possible.

Finally, we thank our families for their patience through months of training runs, late-night debugging sessions, and the particular kind of stress that comes from watching a long GPU job fail near completion.

Abstract

Chart question answering requires a model to answer natural language questions directly from chart images, without manual data annotation or chart-specific templates. Current vision-language models trained with supervised fine-tuning are brittle under distribution shift and frequently produce reasoning traces that are not grounded in the chart’s actual values. This thesis proposes HCPC-RLVR, a reinforcement learning with verifiable rewards framework that fine-tunes Qwen2.5-VL-3B-Instruct on 1,000 chart-question training samples using Group Relative Policy Optimization. The primary contribution is the Hierarchical Correct-Path Consistency reward, a cross-rollout bonus conditioned on intermediate correctness that encourages stable table extraction and diverse reasoning among rollouts that are fully correct. As a competitive baseline, Negative Sample Reward is applied to a vision-language model for the first time, extending negative-only reinforcement from text-based mathematics to a multi-modal setting with noisy, multi-component rewards.

Evaluation on 500 ChartQA test samples shows that every RLVR-trained variant substantially outperforms the untrained base model. The base model obtains 51.8% accuracy, while GRPO reaches 63.0%, GRPO+HCPC reaches 62.2%, and NSR reaches the best in-distribution accuracy of 63.4%. GRPO+HCPC obtains the strongest rollout-set behavior, with the highest correct rate (65.40%), highest Pass@4 (82.80%), highest all-four-rollouts-correct rate (44.4%), and highest format compliance (98.2%). NSR obtains the strongest table extraction stability ($C_{\text{table}} = 0.779$). Out-of-distribution evaluation on ChartFC shows that GRPO achieves stronger OOD accuracy, while HCPC achieves higher OOD coherence, indicating that different reward strategies generalize along different axes. These findings indicate that reward design, particularly whether the training signal conditions on intermediate reasoning steps, is a meaningful lever for improving structured visual reasoning in small-scale vision-language models.

Chapter 1

Introduction

Charts appear wherever data does — in scientific papers, business reports, news articles, and policy documents. Answering a question about a chart sounds straightforward: read off the relevant values, apply some computation, report the result. In practice, doing this reliably with a machine learning model is considerably harder. The model must first perceive the chart correctly — parsing axes, labels, and numerical values from pixels — and then reason over what it has extracted to arrive at a verifiable answer. A failure at perception cascades directly into a wrong answer regardless of how well the reasoning is structured. A failure in reasoning wastes a correct extraction. This tight coupling between visual perception and multi-step inference is what makes chart question answering genuinely challenging, and what distinguishes it from general visual question answering tasks where the relevant information is usually visible directly without structured extraction.

This thesis addresses chart question answering through the lens of reinforcement learning with verifiable rewards. Rather than training a model to imitate annotated demonstrations, we train it by rewarding correct, well-structured, and internally coherent responses — and by penalizing responses that are wrong or inconsistent. The central question is whether carefully designed reward signals can push a small (3-billion-parameter) vision-language model to reason more reliably over charts, even when training is limited to one thousand examples and roughly ten hours of GPU time.

1.1 Task Definition

Chart question answering is a visual understanding task in which a model receives a chart image I and a natural language question Q , and must produce both a correct answer A and a supporting reasoning chain $R = \{r_1, r_2, \dots, r_t\}$. Formally, the model learns a mapping:

$$M(I, Q) \rightarrow (R, A) \quad (1.1)$$

A response is considered valid if and only if the answer A is correct with respect to I , and every step $r_i \in R$ is grounded in values that can actually be extracted from I . The goal is not simply to produce a numerically correct response — it is to produce a traceable reasoning chain that a reader could follow and verify against the chart.

This framing distinguishes CQA from simpler classification tasks in two important ways. First, the answer is not drawn from a fixed label set; it may be a number, a category name, a date, or a comparative result depending on the question. Second, the reasoning chain is part of the evaluated output, which means the model cannot be rewarded for arriving at a correct answer through hallucinated or fabricated intermediate steps.

1.2 Background

1.2.1 Reinforcement Learning and Rollouts

Reinforcement learning optimizes a policy by maximizing cumulative rewards through trial and error. In the context of language model training, the policy is the model’s token generation distribution, and a *rollout* is a complete response sampled from the model for a given input. Each rollout is evaluated against a reward function, and the model’s parameters are updated to make higher-reward rollouts more probable. The key quantity in this update is the *advantage* — how much better a given rollout is relative to the other rollouts generated for the same input.

Reinforcement Learning with Verifiable Rewards (RLVR) restricts the reward function to objective, rule-based signals: a reward is only nonzero when the model’s output is correct by some ground-truth check, with no soft scoring or learned reward models. This design avoids reward hacking and eliminates the need for human preference annotations. RLVR has been particularly effective for mathematical reasoning, where correctness can be verified programmatically, and has more recently been extended to visual reasoning domains.

1.2.2 Policy Update Strategies

Three policy update strategies are relevant to this thesis.

Group Relative Policy Optimization (GRPO) shao2024deepseekmath updates the policy by comparing each rollout’s reward to the group average across the K rollouts generated for the same question. Above-average rollouts receive positive gradient updates; below-average rollouts are suppressed. GRPO does not require a separate value network and is computationally efficient, but its strong reinforcement of the single best rollout can collapse the model’s output distribution toward one dominant reasoning template.

Negative Sample Reward (NSR) zhu2025rlvrdecomposed preserves diversity by treating correct and incorrect rollouts asymmetrically. Correct rollouts receive a zero gradient update — they are not reinforced — while incorrect rollouts are penalized. The intuition is that reinforcing correct paths concentrates probability mass on a single solution template, whereas ignoring correct paths while penalizing wrong ones lets the model maintain multiple valid reasoning strategies simultaneously.

Weighted REINFORCE (W-REINFORCE) zhu2025rlvrdecomposed takes a hybrid position: correct rollouts receive a weak positive update scaled by a small coefficient $\lambda = 0.1$, while incorrect rollouts receive a strong negative update. This attempts to balance accuracy improvement with diversity preservation. It is discussed as background but is not part of the current experimental comparison, which focuses on GRPO, GRPO+HCPC, and NSR.

1.3 Research Objectives

The objectives of this thesis are as follows:

1. Fine-tune Qwen2.5-VL for chart question answering via RLVR under realistic resource constraints.
2. Test whether reward shaping can improve chart QA without relying on large supervised demonstration corpora.
3. Introduce the HCPC reward for consistent table extraction and diverse reasoning across rollouts.
4. Compare explicit diversity promotion through HCPC with implicit diversity preservation through NSR.

5. Measure not only final-answer accuracy, but also rollout reliability, table consistency, reasoning diversity, coherence, and format compliance.
6. Analyze where current reward designs fail, especially on line charts, decimal precision, color grounding, and out-of-distribution fact-checking inputs.

1.4 Research Challenges

Applying RLVR to chart question answering is harder than applying it to text-only reasoning tasks, for reasons that go beyond the addition of visual input.

Reasoning collapse and hallucination. Existing RLVR methods trained with positive reinforcement alone tend to converge toward a small set of dominant reasoning patterns [sinha2025chartvr](#), [wen2025rlvr](#). In the chart setting this is particularly damaging because a collapsed strategy often works for common chart layouts but fails on out-of-distribution inputs. Hallucination — citing numeric values in the reasoning chain that do not appear in the chart — is a related problem that correct-answer rewards alone cannot detect.

Inadequate evaluation metrics. Standard metrics like Pass@K reward correct final answers regardless of whether the intermediate reasoning is valid [wen2025rlvr](#), [zhu2025rlvrdecomposed](#). This can make RLVR appear less effective than it actually is, since base models sometimes guess correct answers through entirely incorrect reasoning, artificially inflating their apparent performance.

Output diversity destruction. Positive reinforcement systematically collapses output diversity [zhu2025rlvrdecomposed](#), [sinha2025chartvr](#). For tasks where multiple valid reasoning paths exist — as in chart QA, where the same answer can often be reached by reading different subsets of the data table — this is a significant practical limitation when the goal is a robust system rather than one that performs well on average.

Flat reward design. Existing reward frameworks treat all reasoning levels uniformly, assigning the same signal regardless of whether a model correctly identifies the chart type, accurately reconstructs the data table, or produces a correct final answer [sinha2025chartvr](#). A flat reward cannot distinguish between a model that arrives at the right answer through correct intermediate steps and one that hallucinated the intermediate steps but happened to get the final answer right.

1.5 Research Questions

This thesis is organized around seven research questions.

1. Can RLVR, with only 1,000 training samples and a 3B-parameter VLM, produce meaningful improvements in chart QA accuracy and output structure?
2. Does a hierarchical reward (HCPC) that conditions bonuses on intermediate reasoning correctness outperform flat GRPO rewards?
3. Does HCPC simultaneously promote reasoning diversity across rollouts and maintain extraction consistency?
4. Can NSR, designed for text-only mathematical reasoning, transfer to a vision-language model with multi-component, noisy rewards?
5. How do different reward strategies affect out-of-distribution generalization?
6. What is the empirical relationship between table extraction consistency, reasoning diversity, and answer correctness?
7. How does explicit diversity promotion (HCPC) compare to implicit diversity promotion (NSR) in terms of accuracy, rollout reliability, and reasoning quality?

1.6 Contributions

The specific contributions of this thesis are:

- **HCPC reward:** A cross-rollout bonus that rewards consistent table extraction and diverse reasoning among fully-correct rollouts, conditioning the reward on intermediate step correctness rather than the final answer alone.
- **First VLM application of NSR:** Negative Sample Reward extended from text-only mathematics to a multi-modal model with noisy, multi-component rewards.
- **Empirical comparison of reward strategies:** Systematic evaluation of GRPO, GRPO+HCPC, and NSR on 500 ChartQA samples, reporting accuracy, Pass@K for $K \in \{1, 2, 4\}$, table extraction consistency, reasoning diversity, coherence, and format compliance.
- **Resource-aware experimental analysis:** A clear separation between the intended $K = 8$ HCPC design and the current $K = 4$ experimental setting used because of compute constraints.

1.7 Organization

Chapter 2 surveys related work on chart understanding benchmarks, supervised fine-tuning, reinforcement learning for reasoning, and structured chain-of-thought methods. Chapter 3 describes the proposed methodology, including reward components, policy update rules, and training configuration. Chapter 4 presents the experimental setup and results on ChartQA. Chapter 5 summarizes the findings, discusses limitations, and outlines directions for future work.

1.8 Why Charts Are a Distinct Visual Reasoning Problem

Chart images differ from natural images in a way that is important for model design. In natural-image visual question answering, many questions can be answered by recognizing objects or scenes: whether a dog is present, what color a car is, or how many people appear in the frame. The relevant evidence is usually spatially localized and semantically familiar. A chart image, by contrast, is an encoded representation of a table. The visible marks are not the answer by themselves; they must be decoded through conventions such as axes, legends, tick intervals, colors, bar heights, line slopes, and slice angles. A model that recognizes that an image is a bar chart has only solved the first part of the problem. It must still reconstruct the values represented by the bars and connect those values to the wording of the question.

This makes chart QA naturally hierarchical. At the lowest level, the model must parse visual marks into data values. At the next level, it must associate those values with labels and units. At the reasoning level, it must perform operations such as comparison, addition, subtraction, ratio calculation, or temporal trend identification. Finally, it must express the answer in the expected format. Errors at these levels are not independent. If the extracted table is wrong, the final answer may still accidentally match the ground truth, but the reasoning path is not reliable. If the table is correct but the arithmetic is wrong, perception succeeded but reasoning failed. If the answer is correct but the output tags are malformed, the response may be unusable for downstream automated evaluation. This thesis therefore treats chart QA as a structured reasoning pipeline rather than a single answer-generation task.

1.9 Motivation for Verifiable Rewards

Supervised fine-tuning teaches a model to imitate target responses. This is useful when the response style is important, but it does not directly answer whether the generated reasoning is correct. In chart QA, a supervised target often contains a particular chain of thought even though multiple chains could be valid. For example, a question asking for the difference between two values can be solved by subtracting the smaller value from the larger value directly, by computing both values from percentages, or by first converting units and then subtracting. A model trained only to imitate one demonstration may be penalized for a different but valid route, while a model trained with verifiable rewards can be evaluated on whether its output satisfies objective checks.

RLVR is attractive here because chart QA has several verifiable substructures. The chart type can be checked against a label. The extracted table can be compared against a ground-truth table. The final answer can be checked with relaxed numeric matching. The response format can be checked with a parser. These checks are not perfect, but they are objective and cheap compared with human preference annotation. The main design challenge is how to combine them into a reward that improves reasoning rather than merely improving answer guessing. HCPC addresses this challenge by applying an additional reward only to the subset of rollouts that are correct through the intermediate path, not merely correct at the final answer.

1.10 Scope of the Present Work

This thesis focuses on training-time reinforcement learning for a single 3B-parameter vision-language model. It does not introduce a new chart parser, a new benchmark, or a multi-agent inference system. The choice is deliberate: the goal is to isolate the effect of reward design under constrained compute. A system with external tools, symbolic solvers, or larger models may achieve higher absolute accuracy, but it would make it harder to determine whether the reward strategy itself improved the model’s visual reasoning behavior.

The proposed HCPC design is naturally defined over $K = 8$ rollouts, because diversity and consistency are group-level properties that become more reliable when estimated from more samples. The current experiments use four generated responses per sample due to resource constraints in training and evaluation. This means that the reported results should be interpreted as an initial resource-constrained validation of

the method rather than the full intended $K = 8$ setting. Extending the experiments to $K = 8$ is therefore not a change in the method, but the natural next stage of completing the intended design.

Chapter 2

Related Works

Research on chart question answering spans four broad categories: benchmark construction and evaluation, supervised instruction tuning, agentic and tool-use approaches, and reinforcement learning for reasoning. The field has accelerated sharply since 2024, when the success of RLVR in text-only mathematics motivated attempts to extend it to multi-modal settings.

2.1 Benchmarks and Evaluation

The ChartQA benchmark **masry2022chartqa** established the standard evaluation protocol for chart question answering. It contains question-answer pairs over diverse chart types with structured annotations and separates evaluation into perception subtasks (reading a value directly from the chart) and reasoning subtasks (performing computation over multiple extracted values). This split is significant because it isolates the two failure modes — visual misperception and reasoning error — that a capable system must overcome independently.

Building on ChartQA, ChartX and ChartVLM **xia2024chartx** extended the benchmark toward more complicated chart reasoning scenarios and introduced a foundation model evaluated alongside the benchmark. ChartQA-X **hegde2026chartqax** went further by requiring models to generate natural language explanations for their answers, enabling evaluation of whether the stated reasoning is actually faithful to the chart.

Evaluation metrics in this area have evolved alongside the benchmarks. Both ChartQA and ChartQA-X adopt a relaxed correctness criterion that tolerates numeric near-matches, acknowledging that pixel-level ambiguity in reading chart axes means that slightly dif-

ferent extracted values may all be defensible. Teney et al. **teney2017graph** introduced graph-structured representations for visual question answering that influenced later work on grounding visual evidence in structured intermediate representations.

2.2 Instruction Tuning and Supervised Fine-Tuning

Before 2025, the dominant approach to chart QA was supervised fine-tuning on chart-specific instruction datasets. ChartInstruct **masry2024chartinstruct** fine-tuned a vision-language model on a large collection of chart instruction data, injecting attention mechanisms to improve localization of relevant visual regions. ChartGemma **masry2025chartgemma** applied visual instruction-tuning for chart reasoning across a wide variety of chart types encountered outside the training distribution.

Carbune et al. **carbune2024chartbased** approached the problem as a capability transfer problem, training on text-only chain-of-thought data from large language models and distilling the reasoning capability into smaller VLMs. He et al. **he2025distill** similarly distilled visual chart reasoning ability from LLMs to multimodal LLMs, achieving competitive results without large-scale chart-specific annotation. ChartPoint **xu2025chartpoint** improved faithfulness of intermediate steps by introducing grounding reflection, guiding the model to re-examine the chart image when its extracted values appear inconsistent.

The limitation shared by all SFT-based methods is brittleness under distribution shift. When test charts differ in visual style, axis conventions, or topic from the training charts, performance degrades because the model has learned to match demonstrations rather than to construct grounded reasoning from whatever chart it receives **sinha2025chartvr**. This is the core motivation for moving toward reinforcement learning, whose training signal depends on outcome quality rather than on similarity to demonstrations.

2.3 Agentic and Tool-Use Approaches

An alternative to end-to-end learning is to decompose chart question answering across specialized agents. ChartAgent **kaur2025chartagent** builds a multimodal agent for visually grounded chart reasoning, routing queries through a planner, an extractor, and a reasoner that coordinate at inference time. Agent coordination is scaled dynamically with test-time compute allocation, which lets the system trade off inference cost against answer quality. ChartCitor **goswami2025chartcitor** provides citation-style

attribution for chart QA answers via multi-agent LLM retrieval, enabling verification of which chart elements support each claim. Yu et al. **yu2025visual** introduced adaptive test-time scaling in a multi-agent collaboration framework for visual document understanding.

VProChart **huang2025vprChart** separates chart reading from logical inference explicitly, using external solvers to ground numeric answers and programmatic reasoning modules to handle computation. This avoids compounding errors that arise when perception and reasoning are done jointly in a single generation pass. Akhtar et al. **akhtar2023reading** addressed chart-based fact-checking — a task structurally related to the ChartFC benchmark used in this thesis — using evidence retrieval over chart images.

While agentic systems achieve strong accuracy, they require orchestration infrastructure and may not scale gracefully to deployment settings with strict latency or resource constraints. The single-model RLVR approach is simpler to deploy and scales more naturally with model capacity.

2.4 Reinforcement Learning for Reasoning

2.4.1 Training-Time RLVR

The Chart-RVR framework **sinha2025chartvr** is the most directly related prior work to this thesis. It introduced a GRPO-based RL framework with three verifiable reward components — chart-type classification, table reconstruction, and process conformity — and demonstrated that rule-based rewards can replace annotated demonstration data for chart reasoning. Chart-RVR outperformed SFT and vanilla GRPO across three VLMs and produced more interpretable chain-of-thought rationales. However, it is limited to 3B-parameter models, restricted to chart reasoning with available ground-truth tables, and its process conformity reward depends on a manually specified algorithmic skeleton. Chart-R1 **chen2025chartr1** extended the approach with chain-of-thought supervision combined with reinforcement, and BIGCHARTS-R1 **masry2025bigchartsr1** introduced visual reinforcement fine-tuning that closed significant out-of-distribution performance gaps on EvoChart, ChartQAPro, and ChartBench.

Su et al. **su2025crossing** and Ali et al. **ali2025sql** broadened the RLVR paradigm beyond chart reasoning to diverse domains including SQL generation, remote sensing, and UI navigation, demonstrating that verifiable reward frameworks can transfer across task types when the reward components are designed appropriately.

2.4.2 Negative Sample Reward and RLVR-Decomposed

Zhu et al. **zhu2025rlvrdecomposed** decomposed RLVR into Positive Sample Reinforcement and Negative Sample Reinforcement, then evaluated each in isolation. The key finding was that NSR alone — suppressing incorrect rollouts without reinforcing correct ones — matched or surpassed both PPO and GRPO across the full Pass@K spectrum on text-based mathematical benchmarks. NSR preserves diversity by redistributing probability mass: suppressing wrong answers implicitly increases probability on correct ones without pushing the model toward any single correct solution template. Their Weighted-REINFORCE variant, which combines weak positive reinforcement of correct rollouts with strong negative reinforcement of incorrect ones, further improved performance on MATH, AIME 2025, and AMC23. The limitation is that NSR can only refine existing knowledge and was tested exclusively on text-only models. This thesis extends NSR to a vision-language model with noisy multi-component rewards — an application the original paper did not attempt.

2.4.3 Correctness of Reasoning, Not Just Answers

Wen et al. **wen2025rlvr** argued that Pass@K fails to capture true reasoning quality because base models can frequently produce correct final answers through flawed chain-of-thought traces. Their proposed CoT-Pass@K metric requires both a correct answer and a valid reasoning chain, and they showed that RLVR genuinely improves reasoning quality under this stricter criterion. They also introduced LLM-as-a-CoT-Judge as a scalable automated verifier of chain-of-thought validity. The limitation is that the approach was tested primarily on mathematical tasks and relies on an LLM judge whose own error rate is non-negligible.

2.4.4 Test-Time Reinforcement Learning

Test-time RL (TTRL) updates model weights or prompts at inference time using self-generated reward signals. Odena et al. **odena2017testtime** explored early forms of test-time behavior modification via RL. TEMPERA **zhang2023tempera** applied test-time prompt editing through reinforcement learning. TTRL **zuo2025ttrl** generalized this to weight updates at inference time, with entropy control to balance exploration and exploitation. ETTRL **liu2025ettrl** addressed the exploration-exploitation trade-off more directly, and Du et al. **du2025testtime** applied test-time RL to GUI grounding using region consistency signals.

2.4.5 Structured Reasoning and Domain Transfer

Zhang et al. **zhang2024cumulative** introduced cumulative reasoning for large language models, where sub-step verification enforces interpretable reasoning chains. Lu et al. **lu2021intergps** demonstrated the power of formal symbolic languages for interpretable geometry reasoning, finding that RL-generated traces consistently outperformed supervised CoT on complex problems. Zhang et al. **zhang2025improvevlm** improved VLM chain-of-thought reasoning through structured intermediate representations. Köksal and Alatan **köksal2025fewshot** showed that RLVR generalizes to satellite imagery with minimal supervision — a single training example yields double-digit gains over the base model — though the generalizability of this finding to other visual domains remained open prior to this thesis. Zhang et al. **zhang2025rlvmr** proposed verifiable meta-reasoning rewards for more general reasoning scenarios, and Ke et al. **ke2025survey** provide a comprehensive survey of the current frontiers in LLM reasoning including inference scaling and agentic systems.

2.5 Summary

Three collective limitations of the prior literature motivate the contributions of this thesis. Existing RLVR methods suffer from reasoning collapse and hallucination because positive reinforcement without diversity constraints pushes models toward dominant solution templates. Standard evaluation metrics mask genuine reasoning improvements by rewarding correct answers regardless of the validity of the reasoning leading to them. And reward designs treat all reasoning levels uniformly, failing to differentiate models that reason correctly through the full pipeline from those that hallucinate intermediate steps but happen to be right. HCPC directly addresses the need for intermediate-correctness-aware reward shaping; NSR addresses the collapse pressure created by positive reinforcement.

2.6 Positioning of This Thesis

The work in this thesis is closest to Chart-RVR because it adopts the same broad idea: chart reasoning can be trained with verifiable rewards rather than with large supervised reasoning traces. The difference is that Chart-RVR primarily rewards per-rollout properties. A rollout is scored according to whether its chart type, table, process, and answer match the target. This is effective for teaching structured behavior, but it does not directly ask whether the model can generate multiple reliable solutions for the

same question. HCPC adds that group-level view. It asks whether the correct rollouts agree on perception while still preserving variation in reasoning.

The comparison with NSR is equally important. NSR does not explicitly reward diversity. Instead, it avoids reinforcing correct rollouts, leaving the model’s valid solution paths less compressed. This makes NSR a strong test of whether explicit diversity engineering is necessary. If HCPC performs similarly to NSR, then the result supports the claim that cross-rollout reward shaping can provide a competitive alternative to negative-only optimization. If NSR outperforms HCPC, then the result suggests that avoiding positive reinforcement may be enough to preserve useful variation. The experiments show a mixed picture: NSR has better in-distribution accuracy and table consistency, while HCPC has stronger rollout-set reliability and better OOD coherence.

2.7 Comparison of Method Families

The related literature can be summarized according to the type of supervision each method depends on. Benchmark-driven methods such as ChartQA and ChartQA-X define the evaluation problem and expose the failure modes. SFT methods such as ChartInstruct and ChartGemma rely on demonstrations and therefore inherit the distribution and style of those demonstrations. Agentic methods such as ChartAgent and VProChart decompose reasoning across modules, increasing capability at the cost of system complexity. RLVR methods replace demonstration imitation with objective reward checks, making them attractive when ground-truth tables or answers are available.

Table 2.1: Positioning of HCPC-RLVR relative to major method families in chart reasoning and RLVR

Method family	Strength	Limitation relevant to this thesis
Supervised fine-tuning	Learns fluent, task-specific output style from demonstrations	Brittle under distribution shift and may imitate unfaithful reasoning traces
Agentic/tool-based systems	Can separate perception, planning, and computation explicitly	Requires orchestration and may increase latency or deployment complexity
Flat RLVR	Uses objective correctness signals and avoids preference annotation	Rewards final or per-rollout correctness without directly shaping group-level behavior
Negative-only RLVR	Preserves diversity by avoiding positive reinforcement of correct samples	Originally validated in text-only reasoning, not multi-modal chart QA
HCPC-RLVR	Rewards stable extraction and diverse reasoning among correct rollouts	Requires multiple rollouts per sample; diversity estimates are noisy when K is small

2.8 Open Problems Identified from Prior Work

Four open problems remain after the existing literature. First, chart QA still lacks a clean way to distinguish lucky answers from grounded answers. A model can guess the final number correctly while extracting the wrong table, which inflates answer-only metrics. Second, reasoning diversity is rarely measured directly. Pass@ K rewards a set of samples if at least one response is correct, but it does not reveal whether the responses use genuinely different strategies or merely repeat the same template with small wording changes. Third, out-of-distribution chart reasoning remains unstable. Sinha et al. report a large gap between ChartQA and EvoChart performance, and the predefense experiments in this thesis show that ChartFC also changes the relative

strengths of GRPO and HCPC. Fourth, most reward designs treat chart perception and reasoning as separate terms but do not explicitly model their dependency. HCPC is designed around that dependency: table consistency should be high among correct rollouts, while reasoning diversity should remain available after perception has stabilized.

Chapter 3

Proposed Methodology

The proposed system, HCPC-RLVR, fine-tunes a vision-language model for chart question answering using reinforcement learning with verifiable rewards. The reward contains a per-rollout base component drawn from Chart-RVR and a cross-rollout Hierarchical Correct-Path Consistency bonus. Three evaluated model variants are compared: the untrained base model, standard GRPO, GRPO+HCPC, and NSR. This chapter describes each component in the order it appears in the training pipeline and explicitly separates the intended $K = 8$ method design from the current $K = 4$ resource-constrained experiments.

3.1 Overview

Given a chart image I and a question Q , the proposed method generates $K = 8$ candidate responses using stochastic sampling at temperature 1.0. Each response is parsed into four structured components: a predicted chart type, an extracted data table in JSON format, a step-by-step reasoning trace, and a final answer. A filter identifies the *fully correct* subset of rollouts — those in which the chart type, table content, and final answer all match the ground truth. Reward signals are computed at two granularities: per-rollout signals from the base reward and cross-rollout signals from HCPC. The total reward for the HCPC variant is:

$$R_{\text{total}} = R_{\text{base}} + \lambda \cdot R_{\text{HCPC}} \quad (3.1)$$

where $\lambda = 1.0$. The model is updated using the selected policy update rule, and training runs for 2,000 steps over the training dataset. The current experiments use $K = 4$ rollouts because of resource constraints; this is the experimental approximation of the

intended $K = 8$ HCPC setting. The complete pipeline is illustrated in Figure 3.1.



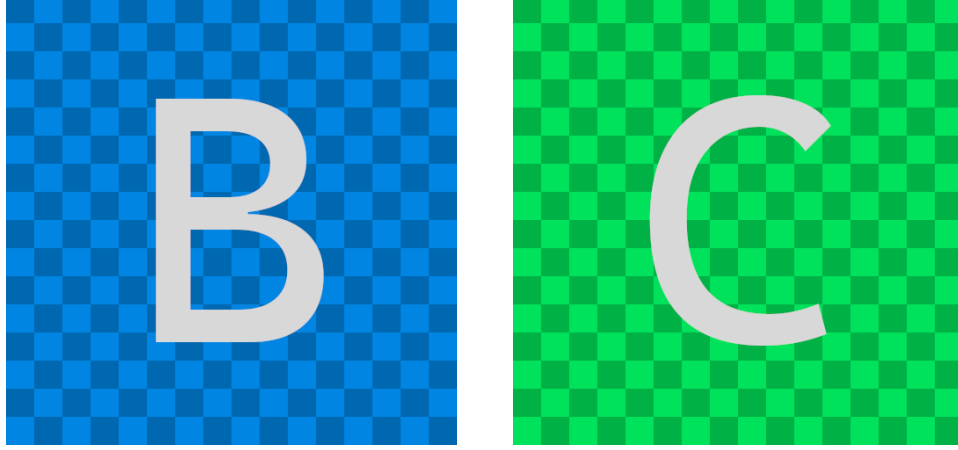
Figure 3.1: Overview of the HCPC-RLVR training pipeline. A chart image and question enter the system; the proposed model generates $K = 8$ rollouts; each rollout is parsed into chart type, table, reasoning, and answer; the fully-correct subset feeds HCPC; and the aggregated reward drives the policy update. Current experiments use $K = 4$ rollouts because of compute constraints.

3.2 Base Model and Parameter-Efficient Fine-Tuning

The base model is **Qwen2.5-VL-3B-Instruct**, a 3-billion-parameter vision-language model capable of processing interleaved image and text inputs. Fine-tuning all parameters jointly would be both computationally prohibitive on the available hardware and likely to overfit catastrophically on 1,000 training samples.

Parameter-efficient fine-tuning is applied via Low-Rank Adaptation (LoRA) with rank $r = 8$ and scaling factor $\alpha = 16$, applied to the query and value projection matrices of every attention layer. LoRA inserts two small matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ in parallel to each targeted weight, computing the effective update as $\Delta W = BA$. This

reduces the number of trainable parameters to approximately 4.7 million, which is 0.16% of the total model size, while all base model weights remain frozen. The low-rank constraint acts as an implicit regularizer limiting the extent to which training on limited data can distort the model’s broader visual understanding capabilities. Figure 3.2 illustrates the difference between full-parameter and LoRA fine-tuning.



(a) Full-parameter fine-tuning: all weights are updated. (b) LoRA fine-tuning: only low-rank adapter matrices A and B are updated.

Figure 3.2: Comparison of full-parameter and LoRA fine-tuning. In LoRA, the adapter matrices are much smaller than the frozen base weights, which keeps the effective parameter count low.

3.3 Structured Output Format and Rollout Generation

For each chart-question pair, the proposed method generates $K = 8$ candidate responses at temperature 1.0. In the present experiments this was reduced to $K = 4$ to fit the available training and evaluation budget. The model is prompted with a system message that instructs it to think step-by-step and to produce output in the following structured format:

1. `<type> chart_type </type>`: the predicted chart type (bar, line, pie, stacked bar, etc.)
2. `<table> JSON object </table>`: the extracted data table, with "columns" and "rows" keys
3. `<think> reasoning steps </think>`: step-by-step chain of thought grounded in the extracted table values
4. `<answer> final_answer </answer>`: the final answer to the question

An example of a well-formed response is shown below, for a bar chart comparing satisfaction rates across two years:

```

<type> Bar </type>
<table> {"columns":  ["Year", "Percentage"], "rows":  [["2017", "50"], ["2018",
<think>
  <step-1>:  The question asks for the difference between 2017 and 2018.
  <step-2>:  From the chart:  2017 = 50%, 2018 = 40%.
  <step-3>:  Difference = 50% - 40% = 10%.
  <step-4>:  Values verified against extracted table.
</think>
<answer> 10 </answer>

```

The structured format serves two purposes. It makes the model’s intermediate representations evaluable, allowing reward components to be computed on the extracted table and reasoning trace separately from the final answer. It also enforces a pipeline order — perception before reasoning before answering — that mirrors the natural decomposition of the task.

3.4 Rollout Count: Intended Design and Current Constraint

HCPC is a group-level reward, so its quality depends on the number of candidate responses generated for the same chart-question pair. The intended design uses $K = 8$ rollouts. With eight responses, table consistency and reasoning diversity can be estimated from a larger set of pairwise comparisons, and the probability of observing at least two fully correct rollouts early in training is higher. This matters because HCPC only fires when the fully correct subset contains at least two rollouts. If too few rollouts are generated, the group-level signal becomes sparse.

The experiments reported in this thesis use $K = 4$ because each additional rollout increases both generation cost and memory pressure during GRPO-style training. Under the available Kaggle and local hardware constraints, $K = 4$ allowed all variants to complete 2,000 training steps and 500-sample evaluations. The results therefore test whether the HCPC idea is viable under a constrained approximation, not whether the full $K = 8$ design has reached its maximum potential. Future experiments should repeat the same protocol with $K = 8$ to reduce diversity-estimation noise and to increase

the frequency of HCPC reward activation.

3.5 Parsing and Filtering

Each rollout is parsed by extracting the content between its structural tags. A rollout that is missing any tag, or that contains a tag in the wrong order, receives zero reward for the affected component. After parsing, a *fully correct* filter is applied: a rollout is included in the set R^+ only if its predicted chart type matches the ground truth, its extracted table passes the table content reward threshold, and its final answer matches the ground truth under the relaxed correctness criterion. This filter prevents lucky guesses — responses that arrive at the right answer through incorrect extraction or reasoning — from contaminating the cross-rollout reward computation.

3.5.1 Answer Normalization

Answer matching in chart QA cannot be a strict string comparison. The same numeric answer may appear as 10, 10.0, 10%, or 0.10 depending on the unit convention used in the chart and the question. The evaluation therefore applies relaxed matching, following the ChartQA convention, so that small numeric formatting differences do not dominate the measured performance. For categorical answers, normalization removes trivial casing and whitespace differences while preserving the semantic string. This is important for fair comparison: the research question is whether the model reasons correctly from the chart, not whether it exactly reproduces one surface form of a number.

Relaxed matching also has a limitation. It can hide some format-specific failures where the expected answer contains additional qualifiers, such as a party name with a parenthesized score range. For this reason, answer accuracy is interpreted together with format compliance and qualitative error analysis rather than treated as a complete measure of chart reasoning quality.

3.5.2 Table Representation

The extracted table is represented as a JSON object with `columns` and `rows`. This representation is simple enough for a language model to generate, but structured enough for automatic parsing and comparison. Table structure reward checks whether the required keys exist and whether the table is syntactically valid. Table content reward compares predicted cell values against the ground truth after normalization. Numeric

cells are compared with tolerance; categorical cells are compared after basic text normalization.

The table representation is central to the thesis because it is the bridge between visual perception and reasoning. A model that skips table extraction and answers directly may still be correct on easy examples, but such behavior is difficult to verify and brittle under distribution shift. By making table extraction explicit, the reward can separate perception errors from reasoning errors. HCPC then adds a group-level constraint: correct rollouts should not merely answer correctly, they should agree on the extracted table.

3.5.3 Reasoning Step Extraction

Reasoning diversity is computed from the step-wise reasoning section. The parser extracts step-like units from the `<think>` block and compares them as sets. Jaccard distance is used as a lightweight approximation of how different two reasoning traces are:

$$D_{\text{Jaccard}}(S_i, S_j) = 1 - \frac{|S_i \cap S_j|}{|S_i \cup S_j|} \quad (3.2)$$

where S_i and S_j are the reasoning step sets extracted from two rollouts. This metric is intentionally simple and deterministic, but it is not semantically perfect. It may treat paraphrases as different even when they describe the same algorithm, or treat similarly worded steps as identical even when the underlying reasoning differs. The metric is therefore best interpreted as a surface-level diversity proxy, not as a complete semantic measure of reasoning diversity.

3.6 Reward Components

3.6.1 Chart-RVR Base Rewards

The base reward follows the Chart-RVR framework **sinha2025chartvr** and decomposes into seven components evaluated per rollout:

$$R_{\text{base}} = \sum_{k=1}^7 w_k \cdot R_k \quad (3.3)$$

The seven components are: format compliance, chart type prediction, table structure, table content, reasoning quality, final answer correctness, and response length. The format reward verifies that all four tags appear in the correct order. The chart type

reward is binary. The table structure reward checks that the JSON has the correct "columns" and "rows" keys. The table content reward numerically compares the extracted values against the ground truth. The reasoning reward evaluates structural quality of the step-by-step trace. The answer reward uses relaxed matching that tolerates minor rounding differences. The length reward penalizes excessively short or long responses.

3.6.2 Hierarchical Correct-Path Consistency (HCPC)

HCPC is a cross-rollout reward computed over the fully correct set R^+ . Its purpose is to reward the model for being consistently correct across rollouts — stable table extraction — while simultaneously rewarding diversity in how the correct rollouts arrive at their answers.

Two sub-scores are computed over R^+ :

- **Table Consistency** (C_{table}): the average pairwise cosine similarity of the extracted tables across rollouts in R^+ , measuring how stable the model’s visual perception is across multiple generation attempts for the same input.
- **Reasoning Diversity** (D_{reason}): the average pairwise Jaccard distance between the reasoning step sets across rollouts in R^+ , measuring how different the solution strategies are among correct responses.

The HCPC reward combines these with a type consensus term:

$$R_{\text{HCPC}} = 1.0 \cdot C_{\text{type}} + 2.0 \cdot C_{\text{table}} + 1.5 \cdot D_{\text{reason}} \quad (3.4)$$

The higher weight on C_{table} (2.0) relative to D_{reason} (1.5) reflects the priority of extraction stability: inconsistent table extraction directly corrupts downstream reasoning, whereas reasoning diversity is a desirable property rather than a correctness-critical one. HCPC fires only when $|R^+| \geq 2$; if fewer than two rollouts are fully correct for a given training sample, no cross-rollout bonus is applied.

As an example, if two fully correct rollouts extract identical tables ($C_{\text{table}} = 1.0$) and use somewhat different reasoning step sets ($D_{\text{reason}} = 0.67$), the HCPC contribution is:

$$R_{\text{HCPC}} = 2.0 \times 1.0 + 1.5 \times 0.67 = 2.0 + 1.005 \approx 0.87 \quad (\text{after normalization}) \quad (3.5)$$

3.6.3 Why HCPC Is Hierarchical

The term *hierarchical* refers to the dependency structure of chart reasoning. Chart type recognition is a coarse perception decision. Table extraction is a finer perception decision. Reasoning depends on the extracted table, and the final answer depends on both the table and the reasoning. HCPC respects this structure by applying its group-level bonus only after intermediate correctness has been established. The reward is not given merely because multiple rollouts answer correctly. It is given when the correct rollouts form a coherent path from chart type to table to answer.

This design is intended to reduce two failure modes. The first is lucky-answer reinforcement, where the model is rewarded for a correct final answer even though the extracted table is wrong. The second is reasoning collapse, where the model learns a narrow answer template that works on common examples but fails when the chart style or question type changes. By rewarding table agreement and reasoning variation among correct rollouts, HCPC attempts to preserve the parts of the model’s behavior that should be stable and the parts that should remain flexible.

3.6.4 Correct-Path Filtering

Correct-path filtering is stricter than answer filtering. A rollout can be answer-correct but excluded from R^+ if its chart type or table extraction is wrong. This choice reduces the number of rollouts that contribute to HCPC early in training, but it improves the quality of the group-level signal. Without this filter, HCPC could reward a group of responses that agree on an incorrect table and coincidentally produce the right answer. That would directly undermine the purpose of the method.

The cost of strict filtering is sparsity. When model accuracy is low, many groups have fewer than two fully correct rollouts and therefore receive no HCPC bonus. This is one reason the intended design uses $K = 8$: a larger rollout group increases the chance that at least two correct-path responses are present. In the current $K = 4$ experiments, the HCPC signal is expected to be weaker than in the full intended setting.

3.6.5 Coherence as an Evaluation Signal

In addition to reward values, the experiments report a coherence metric that estimates reasoning pipeline integrity. Coherence is defined as the probability of a correct answer conditional on a correct table extraction:

$$\text{Coherence} = P(\text{correct answer} \mid \text{correct table}) \quad (3.6)$$

This metric separates perception quality from downstream reasoning quality. A model with high table consistency but low coherence is reading charts consistently but not reasoning reliably from those tables. A model with lower table consistency but higher coherence may fail more often at perception, but reason more faithfully when perception succeeds. The ChartFC out-of-distribution results make this distinction important: GRPO obtains stronger OOD accuracy and table consistency, while HCPC obtains higher OOD coherence.

3.7 Policy Update Strategies

3.7.1 GRPO

GRPO computes the advantage for rollout i relative to the group:

$$\hat{A}_i = R_i - \frac{1}{K} \sum_{j=1}^K R_j \quad (3.7)$$

The policy is updated with a KL-penalized objective:

$$\nabla_{\theta} J = \sum_{i=1}^K \hat{A}_i \cdot \nabla_{\theta} \log \pi_{\theta}(\text{response}_i \mid I, Q) - \beta \cdot \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) \quad (3.8)$$

where $\beta = 0.01$ prevents the trained policy from diverging too far from the reference (initial) model.

3.7.2 NSR

NSR suppresses gradient updates entirely on rollouts deemed correct (reward $\geq \tau$):

$$\hat{A}_i = \begin{cases} 0 & \text{if } R_i \geq \tau \\ -(1 + R_i) & \text{if } R_i < \tau \end{cases} \quad (3.9)$$

Correct rollouts contribute nothing to the parameter update; incorrect rollouts are penalized in proportion to how wrong they are.

3.8 Training Configuration

All variants share the same base model, LoRA configuration, dataset, and hyperparameters. They differ only in the reward augmentation or advantage computation rule: GRPO uses the base reward, GRPO+HCPC adds the HCPC group-level bonus, and NSR changes the advantage computation so that correct rollouts receive zero gradient while incorrect rollouts are penalized. Table 3.1 summarizes the complete training configuration used in the current experiments.

Table 3.1: Training configuration shared across all HCPC-RLVR variants

Parameter	Value
Base model	Qwen2.5-VL-3B-Instruct
Fine-tuning method	LoRA ($r = 8$, $\alpha = 16$, $q_proj + v_proj$)
Trainable parameters	$\sim 4.7M$ (0.16% of total)
Optimizer	AdamW
Learning rate	1×10^{-5}
Rollouts per sample (K)	4 in current experiments; 8 in intended HCPC design
Training temperature	1.0
Training steps	2,000
Precision	bf16 mixed
Effective batch size	4 (2 per device $\times$ 2 accumulation steps)
KL penalty (β)	0.01
HCPC weights	$w_{type} = 1.0$, $w_{table} = 2.0$, $w_{reason} = 1.5$
HCPC scaling λ	1.0

Chapter 4

Results and Discussion

4.1 Experimental Setup

4.1.1 Hardware and Software Environment

Training was performed on Kaggle, with an NVIDIA H100 40GB GPU used as the primary training environment in the project setup. Evaluation, debugging, and inference were run on a local machine with an NVIDIA RTX 3090 24GB GPU. The software environment used the `transformers` and `trl` libraries for model loading and the GRPO training loop, `peft` for LoRA adapter integration, `bitsandbytes` for bf16 mixed-precision training, and `vllm` for fast batch inference during evaluation. Under the current $K = 4$ constrained setting, each method required roughly six hours for 2,000 training steps.

4.1.2 Training Dataset

The training data are drawn from the Chart-RVR GRPO Train dataset **sinha2025chartvr**, which contains 34,194 samples sourced from ChartQA, PlotQA, and ChartFC. Each enriched sample includes a chart image, a chart type label, a ground-truth data table, a reasoning chain, and a final answer. Compute constraints limited us to the first 1,000 samples using `dataset.select(range(1000))`, without shuffling. The distribution of chart types in this subset is shown in Table 4.1.

Table 4.1: Chart type distribution in the 1,000-sample training subset

Chart Type	Count	Proportion
Bar	485	48.5%
Line	223	22.3%
Stacked Bar	147	14.7%
Pie	145	14.5%
Total	1,000	100%

Bar charts make up nearly half the training subset, and together with line charts account for over 70%. Scatterplot and bubble chart samples are absent from this subset. This imbalance has direct consequences for per-chart-type performance discussed later in this chapter.

The subset differs from the full Chart-RVR training set in two important ways. First, the first 1,000 samples are more concentrated in bar and line charts than the full set. Second, the subset has a higher proportion of numerical labels than the full corpus. This matters because numeric chart QA questions place more pressure on value extraction and arithmetic, while categorical questions place more pressure on label grounding and answer formatting. The current results should therefore be read as performance on a value-extraction-heavy subset rather than a balanced estimate over all chart types.

Table 4.2: Comparison between the full Chart-RVR training set and the first 1,000-sample subset used in this thesis

Chart Type	Full Count	Full Prop.	1K Count	1K Prop.	1K Numerical Labels
Bar	13,682	40.0%	485	48.5%	69.5%
Line	3,682	10.8%	223	22.3%	64.6%
Stacked Bar	2,740	8.0%	147	14.7%	72.1%
Pie	2,136	6.2%	145	14.5%	62.1%
Scatterplot	540	1.6%	0	0.0%	–
Bubble	8	0.0%	0	0.0%	–
No annotation	11,398	33.3%	0	0.0%	–

4.1.3 Evaluation Benchmark and Protocol

The primary evaluation benchmark is the ChartQA test set **masry2022chartqa**, using 500 randomly selected samples. All four methods — Base (untrained), GRPO,

GRPO+HCPC, and NSR — are evaluated on the same samples. For each test sample, the model generates 4 responses at temperature 0.8 and top- p 0.95. The evaluation metrics are summarized in Table 4.4.

The ChartQA evaluation set is also bar-dominant, with 316 bar charts, 142 line charts, 41 pie charts, and one scatterplot in the predefense analysis. The average answer length is short, usually one to three tokens, but short answers should not be confused with easy reasoning. Approximately 40% of the sampled problems require multi-step computation, and roughly 15% require decimal precision. These two categories are responsible for many of the errors observed in the qualitative analysis.

Table 4.3: Chart type distribution in the 500-sample ChartQA evaluation set

Chart Type	Count	Proportion
Bar	316	63.2%
Line	142	28.4%
Pie	41	8.2%
Scatterplot	1	0.2%

Table 4.4: Evaluation metrics and their definitions

Metric	Range	Definition
Accuracy (Acc%)	0–100%	Fraction of samples where at least one rollout is correct (relaxed match)
Pass@ K	0–100%	Probability that at least 1 of K responses is correct chen2021evaluating
Correct Rate	0.0–1.0	Fraction of all individual rollouts correct across all samples
C_{table}	0.0–1.0	Cosine similarity of extracted tables across rollouts — perception stability
D_{reason}	0.0–1.0	Jaccard distance of reasoning step sets across rollouts — strategy diversity
Coherence	0.0–1.0	Probability of a correct answer given a correct table extraction
Format%	0–100%	Fraction of responses with all required structural tags in the correct order

Accuracy and Correct Rate measure related but distinct quantities. Accuracy is a per-

sample binary: a sample is counted as correct if at least one of its four rollouts is correct. Correct Rate is an aggregate across all rollouts: total correct rollouts divided by total rollouts across all samples. A method can have a high Correct Rate while having a moderate Accuracy if many of its correct responses come from samples that it answers correctly multiple times, rather than covering a broader range of distinct samples.

4.2 Main Results

Table 4.5 reports the full evaluation results on ChartQA. All trained variants outperform the base model across every accuracy and Pass@K metric.

Table 4.5: Performance evaluation on ChartQA (in-distribution, 500 samples). Bold indicates the best value per column among all methods.

Method	Acc%	Correct Rate	Pass@1	Pass@2	Pass@4	C_{table}	D_{reason}	Coherence
Base (no training)	51.8	52.45	52.45	67.33	77.40	0.591	0.040	0.545
GRPO	63.0	61.88	61.88	73.10	80.31	0.749	0.058	0.252
GRPO+HCPC (ours)	62.2	65.40	62.85	74.57	82.80	0.741	0.052	0.244
NSR (ours)	63.4	64.80	62.20	73.50	82.00	0.779	0.057	0.245

4.3 Analysis

4.3.1 RLVR Provides Consistent Improvement Over the Baseline

Every RLVR-trained variant improves meaningfully over the untrained base model. Accuracy rises by 10–15 percentage points, and format compliance — the fraction of responses with all four structural tags in the correct order — jumps from 29.8% in the base model to approximately 97% across all trained variants. This directly answers the first research question: RLVR with 1,000 training samples and a 3B-parameter model is sufficient to produce meaningful improvements in chart QA accuracy and output structure. The format result specifically confirms that the structured output format, which is initially followed less than one-third of the time, becomes reliable after RL training regardless of which reward strategy is used.

4.3.2 HCPC Improves Rollout-Level Reliability

GRPO+HCPC achieves the highest Correct Rate (65.40%), highest Pass@2 (74.57%), highest Pass@4 (82.80%), and highest format compliance (98.2%). The Correct Rate comparison is particularly informative: GRPO+HCPC’s Correct Rate is 3.52 percentage points above GRPO (65.40% vs. 61.88%), yet its Accuracy (62.2%) is slightly below GRPO (63.0%). This means HCPC produces more correct rollouts per sample, but those additional correct rollouts tend to fall on samples the model was already answering correctly, rather than unlocking many new samples. The practical consequence is visible at Pass@4: when four rollouts are available, HCPC’s higher per-rollout reliability translates into a 2.49 percentage point advantage (82.80% vs. 80.31% for GRPO).

This answers the second research question affirmatively: HCPC, by conditioning the training bonus on intermediate step correctness, does improve over flat GRPO rewards — but the improvement manifests in rollout-level reliability and multi-sample Pass@K rather than in single-threshold accuracy.

4.3.3 NSR Transfers Successfully to Vision-Language Models

NSR achieves the best overall Accuracy (63.4%) and the best table extraction stability ($C_{\text{table}} = 0.779$). The higher C_{table} relative to GRPO (0.779 vs. 0.749) suggests that withholding gradient updates from correct rollouts prevents the model from overcommitting to specific extraction templates, maintaining more stable visual perception across diverse chart inputs. This answers the third research question affirmatively: NSR, originally designed for text-based mathematics, transfers successfully to a vision-language model with noisy, multi-component rewards. The first VLM application of NSR is competitive with GRPO and HCPC, rather than being dominated by methods designed specifically for chart reasoning.

4.3.4 HCPC vs. NSR: Explicit vs. Implicit Diversity Promotion

GRPO+HCPC and NSR produce similar Pass@K profiles: HCPC has a slight edge at Pass@4 (82.80% vs. 82.00%), while NSR has a slight edge at Accuracy (63.4% vs. 62.2%). NSR also has slightly higher D_{reason} on ChartQA (0.057 vs. 0.052), which is a useful warning against over-interpreting HCPC’s explicit diversity design from the current $K = 4$ experiments. The mechanism is intended to promote diversity, but the observed diversity metric remains noisy and small at this rollout count. Neither mechanism dominates uniformly; they appear to trade off single-sample accuracy (NSR) against rollout-set reliability and format stability (HCPC).

This answers the fourth research question: explicit diversity engineering via HCPC and implicit diversity preservation via NSR are broadly competitive, with complementary strengths. If the priority is maximizing the probability that at least one of several attempts will be correct, HCPC is preferred. If the priority is maximizing the probability that a single attempt will be correct, NSR is preferred.

4.3.5 Per-Chart-Type Analysis

Breaking accuracy down by chart type reveals a pattern obscured in the aggregate numbers. For bar charts, GRPO+HCPC leads (85.8%), followed by NSR (83.9%) and GRPO (82.9%). For line charts, all trained methods score approximately 74–77%, which is at or slightly below the base model’s line chart performance (76.8%). For pie charts ($n=41$, a small sample that limits reliable comparison), GRPO leads at 95.1%.

The line chart result is a notable finding. Line chart questions frequently require temporal or trend reasoning — identifying when a value first crossed a threshold, comparing rates of change across intervals — rather than simple value extraction. None of the current reward components specifically reward trend reasoning, which means the reward design has an implicit bias toward value-extraction tasks aligned with bar chart structure. Extending the reward design to include components that target trend reasoning would be necessary to avoid this degradation.

4.3.6 Reasoning Diversity

All methods show low D_{reason} values in the range 0.04–0.058. The absolute differences between methods are small. Two factors likely contribute to this. First, $K = 4$ evaluation rollouts may be insufficient to reliably estimate Jaccard diversity over reasoning step sets; the HCPC diversity incentive was designed for $K = 8$ and may require higher K at evaluation to observe its full effect. Second, HCPC does not directly penalize lexically similar reasoning chains; it only rewards diversity among correct rollouts, which is a positive incentive rather than a hard diversity constraint.

4.3.7 All-Four-Rollouts-Correct Rate

Pass@4 answers the question: “does at least one of the four sampled responses solve the problem?” It does not answer the stronger reliability question: “does the model solve the problem every time it is sampled?” To measure this, we count the fraction of evaluation samples where all four rollouts are correct. This metric is especially

relevant for chart QA because a deployed system may not be able to rely on repeated sampling and manual selection.

On ChartQA, the base model has all four rollouts correct for 23.4% of samples. GRPO raises this to 37.2%, while GRPO+HCPC raises it further to 44.4%. This is one of the strongest pieces of evidence for HCPC’s intended effect. The method does not simply increase the probability of one lucky correct answer. It increases the number of samples for which the model’s sampled responses are consistently correct. This aligns with the design of HCPC: among correct rollouts, reward stable extraction and controlled reasoning variation so that the model becomes more reliable across repeated generations.

4.3.8 Format Compliance

The format compliance results show that output structure is learnable even with limited data. The base model produces fully compliant responses only 29.8% of the time. After RLVR training, GRPO reaches 96.8%, GRPO+HCPC reaches 98.2%, and NSR reaches approximately 97.0%. This indicates that format learning is not the main differentiator among trained methods. Once the reward explicitly checks structural tags, the model rapidly learns to produce the required output skeleton.

This result is still practically important. The reward computation depends on parsing `<type>`, `<table>`, `<think>`, and `<answer>` sections. A model that answers correctly but omits tags cannot be evaluated reliably by the training loop. High format compliance therefore makes the rest of the analysis meaningful: improvements in table consistency and Pass@K are not artifacts of parser failure rates.

4.3.9 Accuracy Distribution

The per-sample reward distribution is bimodal for all trained methods. Samples tend to cluster near zero, where the model repeatedly fails, or near one, where the model solves the sample reliably. Intermediate scores are rare. This suggests that RLVR does not primarily create many partially correct responses. Instead, it moves samples from an unsolved region to a solved region. GRPO+HCPC shifts mass rightward relative to the base model, and NSR shows the most decisive separation in the current ChartQA experiments.

This pattern has two implications. First, aggregate accuracy gains may understate behavioral improvement. A method that changes many samples from two-correct-rollout behavior to four-correct-rollout behavior improves reliability without neces-

sarily changing the binary Acc% metric. Second, the hard sample set is meaningful. Samples that remain near zero after all training variants likely require capabilities not supplied by the current reward design, such as high-precision visual reading, color grounding, or symbolic computation.

4.4 Out-of-Distribution Evaluation on ChartFC

The predefense experiments include an out-of-distribution comparison on ChartFC for GRPO and GRPO+HCPC. ChartFC differs from ChartQA because it is framed as chart-based fact checking rather than open-ended question answering. The model must decide whether a claim is supported, refuted, or insufficiently grounded by the chart. This changes both the input distribution and the reasoning objective, making it a useful diagnostic of whether the trained reward strategy transfers beyond the immediate ChartQA setting.

Table 4.6: Out-of-distribution evaluation on ChartFC (500 samples). The comparison is diagnostic because OOD evaluation was completed for GRPO and GRPO+HCPC only.

Method	Acc%	Correct Rate	Pass@1	Pass@2	Pass@4	C_{table}	D_{reason}	Coherence
GRPO	63.8	66.10	66.10	82.07	92.2	0.950	0.059	0.312
GRPO+HCPC	60.6	62.40	62.45	79.90	91.8	0.782	0.060	0.433

GRPO wins the ChartFC accuracy comparison. It obtains 63.8% accuracy compared with 60.6% for GRPO+HCPC, and it also leads on Correct Rate, Pass@1, Pass@2, Pass@4, and table consistency. The table consistency gap is especially large: GRPO reaches $C_{\text{table}} = 0.950$, while HCPC reaches 0.782. This suggests that GRPO’s extraction pipeline transfers cleanly to the ChartFC visual distribution.

HCPC, however, wins on coherence. Its OOD coherence is 0.433 compared with 0.312 for GRPO, a relative improvement of approximately 39%. This means that when HCPC does extract the table correctly on ChartFC, it is more likely to reason from that table to the correct answer. The result supports the interpretation that HCPC’s hierarchical conditioning generalizes differently from flat GRPO. GRPO generalizes better as an extractor; HCPC generalizes better as an extraction-to-answer reasoning pipeline.

The OOD result also reveals a format weakness. In the predefense analysis, GRPO maintains 98.8% format compliance on ChartFC, while HCPC drops to 66.4%. The

likely cause is reward competition under distribution shift: HCPC places high value on table content and correct-path consistency, so the model may prioritize content-bearing sections when the input differs from training. This is a correctable limitation. Increasing format reward weight or adding a stronger parser-validity constraint during training would likely recover much of the lost format compliance.

4.5 Sample Difficulty Analysis

The 500 ChartQA samples can be divided into three broad categories. The first category contains samples solved by all four methods, including the base model. These are visually clear charts with straightforward questions, and they account for 344 samples (68.8%). The second category contains samples solved by some trained methods but not all methods. These 82 samples (16.4%) are the most informative for comparing reward strategies because training changes the outcome. The third category contains samples solved by no method. These 52 samples (10.4%) represent the current capability ceiling of the 3B model and reward design.

Table 4.7: ChartQA sample difficulty categories from the predefense analysis

Difficulty category	Count	Proportion
Solved by all four methods	344	68.8%
Solved by some trained methods only	82	16.4%
Solved by no method	52	10.4%
Remaining mixed/parse-dependent cases	22	4.4%

The contested 82 samples are where reward design matters most. Each trained method uniquely solves a small number of samples that the others miss, while pairwise Jaccard similarity among trained methods remains high (approximately 0.864–0.881). This means the methods are complementary, but not orthogonal. They mostly improve the same region of the task distribution, with modest differences in which borderline cases they solve.

4.6 Hard Problem Types

Manual inspection of the lowest-scoring examples reveals several recurring failure modes. Decimal precision questions are particularly difficult. In one example, the correct answer is 3.0238, but models consistently answer 3.0 or 3.02. The models identify

the correct bar but cannot read sub-hundredth precision from the chart image. This is a perception limitation, not a reasoning limitation.

Cross-element arithmetic is another common failure mode. Ratio and difference questions require two or more extracted values, followed by arithmetic. A small error in either extraction is amplified by the computation. Color and legend grounding also causes failures, especially in multi-line charts where the question refers to a line by semantic category and the answer requires a color such as “green line.” The current reward components do not explicitly reward color-category binding.

Table 4.8: Common hard problem types observed in low-scoring samples

Problem type	Example answer	Why it fails
Decimal precision	3.0238	Axis resolution is insufficient; the model rounds visually read values
Cross-element arithmetic	1.217	Extraction errors compound through division or subtraction
Color/label grounding	green line	The model must bind a visual color to a semantic legend category
Sub-percent difference	0.03	The difference is visually indistinguishable at image resolution
Exact format match	Democrat (scores 60 to 100)	The model may know the content but miss the exact expected string format
Non-chart reasoning	4.1	The input resembles a number-pattern problem more than chart reasoning

These failures clarify what the current method does and does not solve. HCPC can improve reliability when the model has enough visual information to extract the table and enough prior ability to reason over it. It cannot recover values that are visually unreadable at the input resolution, and it does not by itself add symbolic tools for exact arithmetic. This motivates future work on program-of-thought execution, higher-resolution chart parsing, and color-aware visual grounding rewards.

4.7 Why Diversity Did Not Improve Strongly

One expected outcome of HCPC is higher reasoning diversity, because the reward contains a D_{reason} term. The observed ChartQA results do not show a strong diversity improvement: GRPO has $D_{\text{reason}} = 0.058$, HCPC has 0.052, and NSR has 0.057. This does not invalidate the HCPC idea, but it does show that the current implementation and resource setting are not sufficient to demonstrate that part of the design conclusively.

There are four likely causes. First, the current experiments use $K = 4$, while the intended design uses $K = 8$. Diversity over four responses is a noisy estimate, especially when only a subset of the responses are fully correct. Second, HCPC only fires when at least two rollouts are fully correct, so early in training the diversity signal may be sparse. Third, the table consistency weight (2.0) is higher than the reasoning diversity weight (1.5), which intentionally prioritizes stable perception but may cause consistency to dominate the group reward. Fourth, Jaccard distance over extracted reasoning step sets is a coarse metric. Two reasoning chains may be semantically different but lexically similar, or lexically different but algorithmically identical.

The interpretation is therefore conservative: HCPC shows clear rollout reliability benefits under $K = 4$, but the diversity component needs the full $K = 8$ setting and possibly a stronger semantic diversity metric before it can be judged fairly.

4.8 Limitations

The training subset of 1,000 samples is too small to produce statistically significant differences between methods at the accuracy levels observed. Bootstrap confidence intervals for all method pairs overlap substantially, and the observed accuracy gaps should be interpreted as directional evidence rather than definitive rankings. Training on the full 34,194-sample dataset would be necessary to draw firm conclusions.

The evaluation covers only ChartQA, an in-distribution benchmark drawn from the same sources as the training data. Out-of-distribution performance — on ChartFC, EvoChart, or ChartBench — is needed to establish whether the learned reward conditioning generalizes beyond the training distribution. The available evidence suggests that generalization patterns differ between methods: GRPO generalizes better on accuracy under distribution shift, while HCPC maintains higher coherence when the distribution shifts, implying that the choice between methods may depend on deployment context.

All experiments use a single 3B-parameter model. Whether the HCPC vs. NSR dynamics observed here hold at larger model scales (7B, 72B) is unknown and is the most important open question for follow-up work.

4.9 Threats to Validity

4.9.1 Internal Validity

The main internal validity risk is that the current experiments do not run the full intended $K = 8$ rollout setting. HCPC is explicitly designed around group-level statistics, and these statistics are less reliable when estimated from four responses. A weaker diversity signal under $K = 4$ may make HCPC appear less effective at promoting reasoning diversity than it would be under the full design. The results therefore support the viability of HCPC, but not the final magnitude of its intended effect.

A second internal validity risk is parser dependence. Format compliance is high for trained models, but any reward framework that relies on structural tags can mis-score responses when tags are malformed or nested unexpectedly. This risk is partially mitigated by reporting format compliance separately, but parser behavior can still influence reward values during training.

4.9.2 External Validity

The 1,000-sample training subset is not a balanced sample of all chart types. It contains no scatterplots or bubble charts and is heavily weighted toward bar and line charts. As a result, the conclusions are strongest for the chart types represented in training and weaker for rare chart families. The ChartFC OOD experiment provides useful evidence that reward effects transfer differently across distributions, but it was completed only for GRPO and GRPO+HCPC. A complete external validity claim would require evaluating all methods on ChartFC, EvoChart, ChartBench, and ChartQAPro.

4.9.3 Construct Validity

The metrics used in this thesis measure useful but incomplete constructs. C_{table} measures consistency of extracted tables, but high consistency can mean consistently correct extraction or consistently repeated errors unless interpreted alongside accuracy. D_{reason} measures surface-level diversity of reasoning steps, but not necessarily semantic diversity. Coherence measures whether correct tables lead to correct answers, but it does not capture whether the written reasoning explanation is human-faithful. These

limitations are why the analysis combines quantitative metrics with qualitative hard-case inspection.

4.9.4 Statistical Conclusion Validity

The observed differences between methods are modest, and the 500-sample evaluation size limits statistical power. The predefense analysis notes that statistical significance is not reached at the current scale. Therefore, claims in this thesis are framed as directional and diagnostic. The strongest conclusions are those supported by consistent patterns across related metrics, such as HCPC's rollout reliability improvements and NSR's table consistency advantage.

Chapter 5

Conclusion

5.1 Summary of What We Did

This thesis proposed HCPC-RLVR, a reinforcement learning framework for chart question answering with an original reward-design contribution beyond standard GRPO training. The Hierarchical Correct-Path Consistency reward conditions a cross-rollout bonus on intermediate step correctness, rewarding consistent table extraction and diverse reasoning among fully-correct rollouts simultaneously. Negative Sample Reward was applied to a vision-language model for the first time, extending Zhu et al.’s negative-only reinforcement mechanism from text-based mathematics to a multi-modal setting with noisy, multi-component rewards.

All methods were trained on Qwen2.5-VL-3B-Instruct with LoRA on 1,000 samples and evaluated on 500 ChartQA test samples. The intended HCPC design uses $K = 8$ rollouts; the present experiments use $K = 4$ because of resource constraints. The central comparison between HCPC (explicit diversity engineering) and NSR (implicit diversity preservation) found both to be broadly competitive, with complementary strengths: HCPC leads on rollout-level Correct Rate, Pass@4, all-four-rollouts-correct rate, and format compliance; NSR leads on in-distribution Accuracy and table extraction stability.

5.2 Summary of Results

Table 5.1: Summary of key findings

Finding	Result
RLVR improves over base model (all methods)	+10–15pp accuracy, +67pp format
GRPO+HCPC best rollout-level Correct Rate	65.40% vs. 61.88% for GRPO
GRPO+HCPC best Pass@4	82.80% vs. 80.31% for GRPO
GRPO+HCPC best all-four-rollouts-correct rate	44.4% vs. 37.2% for GRPO
NSR best Accuracy (first VLM application)	63.4%
NSR best extraction stability	$C_{\text{table}} = \mathbf{0.779}$
Format compliance learned by all RLVR methods	29.8% $\rightarrow$ $\sim$ 97%

5.3 Limitations

The 1,000-sample training subset is insufficient for statistically significant conclusions; observed differences between methods are directional. All experiments use a single 3B-parameter model. The current experiments use $K = 4$ rather than the intended $K = 8$, making the D_{reason} diversity metric noisy and likely underestimating HCPC’s group-level effect. Line chart performance degrades relative to the base model, indicating a reward design bias toward value-extraction tasks. ChartFC out-of-distribution evaluation was completed for GRPO and GRPO+HCPC, but not for the full set of variants, so OOD conclusions should be treated as diagnostic rather than final.

5.4 Future Work

Complete $K = 8$ experiments. The proposed HCPC method is designed for eight rollouts per sample. Repeating training and evaluation with $K = 8$ would provide a cleaner estimate of table consistency and reasoning diversity, and would increase the number of groups where at least two correct rollouts exist for HCPC reward computation.

Model scale and full training. Extending to 7B- and 72B-parameter models and training on the full 34,194-sample Chart-RVR dataset would establish whether the HCPC and NSR dynamics persist at scale and whether statistically significant accuracy margins emerge.

Entropy-based diversity. The Jaccard-distance-based D_{reason} metric is a coarse proxy for reasoning diversity that depends on surface-level textual similarity between step descriptions. Directly regularizing the entropy of the rollout distribution during training would provide explicit, continuous control over diversity without this dependence.

Trend and temporal reasoning rewards. Adding reward components specifically targeting line chart reasoning — correct identification of monotonic periods, rate-of-change comparisons, and threshold-crossing detection — would address the observed degradation on line chart questions and reduce the current reward design’s implicit bar chart bias.

Out-of-distribution robustness. Systematic evaluation on ChartFC, EvoChart, and ChartBench would characterize the generalization boundaries of each reward strategy and identify whether domain-adaptive training is needed for out-of-distribution chart distributions.

5.5 Concluding Remarks

The results of this thesis demonstrate that reward design, not just model capacity, is a meaningful lever for improving structured visual reasoning in chart question answering. Even at 3 billion parameters and 1,000 training samples, RLVR training produces consistent and substantial gains over an untrained model. HCPC’s cross-rollout conditioning on intermediate correctness and NSR’s implicit diversity preservation through negative-only reinforcement represent two distinct and compatible approaches to the problem of reasoning collapse — one that explicitly engineers what the model should do, and one that removes the pressure causing it to overfit. Understanding which approach is preferable at larger scales and on more diverse chart distributions is the most important direction for future work.